

1. Definition of Functional Interface

A Functional Interface in Java is an interface that contains exactly one abstract method. It can have any number of default methods, static methods, and methods inherited from the `Object` class. These interfaces are also known as SAM (Single Abstract Method) interfaces.

Key Points:

- Single Abstract Method: Only one abstract method is allowed.
- Default and Static Methods: Can contain multiple default and static methods.
- `@FunctionalInterface` Annotation: Optional but recommended; it ensures the interface has only one abstract method and causes a compile-time error if violated.
- Purpose: Forms the foundation for lambda expressions and method references in Java 8+.

Syntax:

java

```
@FunctionalInterface
interface MyFunctionalInterface {
    void abstractMethod(); // Single abstract method
}
```

2. Examples of Functional Interfaces

Example 1: Built-in - `Runnable` (from `java.lang`)

java

```
@FunctionalInterface
public interface Runnable {
    void run(); // Single abstract method
}
```

Usage with Lambda:

java

```
Runnable task = () -> System.out.println("Running in a thread");
new Thread(task).start();
```

Example 2: Built-in - Comparator<T> (from java.util)

java

```
@FunctionalInterface
public interface Comparator<T> {
    int compare(T o1, T o2); // Single abstract method
    // many default and static methods
}
```

Usage with Lambda:

java

```
Comparator<String> byLength = (s1, s2) -> Integer.compare(s1.length(), s2.length());
List<String> list = Arrays.asList("apple", "banana", "cherry");
Collections.sort(list, byLength);
```

Bonus: Custom Functional Interface Example

java

```
@FunctionalInterface
interface MathOperation {
    int operate(int a, int b); // Single abstract method
}

// Usage:
MathOperation addition = (a, b) -> a + b;
System.out.println(addition.operate(5, 3)); // Output: 8
```

Common Built-in Functional Interfaces in java.util.function:

Interface	Abstract Method	Purpose
Predicate<T>	boolean test(T t)	Conditional checks
Function<T,R>	R apply(T t)	Transform input to output
Consumer<T>	void accept(T t)	Consume/process value
Supplier<T>	T get()	Provide/return a value

These interfaces power Java's functional programming features, enabling cleaner, more expressive code with lambdas and streams.